PLATEFORME DE LOCATION DE VOITURES EN MICROSERVICES

Rentify



ARCHITECTURE WEB
DISTRIBUÉE
CONCEPTION EN DDD

VALIDATION 1 - MODULE AWD 2026-2027

ÉQUIPE 4SSA9 - G3



RENTIFY - VALIDATION 1 AWD

PROBLÉMATIQUE

POURQUOI RENTIFY ?

Un parc de véhicules, des réservations, des contrats, des paiements et des infractions : dans un **monolithe**, ces domaines sont couplés et difficiles à faire évoluer.

Notre réponse : une plateforme **en microservices**, conçue en **DDD**. Un contexte métier correspond à un microservice, avec sa propre base de données.

FONCTIONNALITÉ DISTINCTIVE

Quand une infraction est déclarée sur une voiture (plaque, date et heure), le système retrouve le **contrat actif**, identifie le **conducteur** et crée la pénalité.

RENTIFY - VALIDATION 1 AWD

FONCTIONNALITÉS

ACTEURS ET CAS D'UTILISATION GÉNÉRAL



CLIENT

- Créer son compte, gérer son profil et son permis
- Rechercher un véhicule disponible
- Réserver, modifier, annuler
- Consulter contrats et factures, payer

AGENT DE LOCATION

- Gérer le parc
- Ouvrir et clôturer un contrat (état des lieux)
- Déclarer une infraction
- Planifier la maintenance

ADMINISTRATEUR

- Gérer véhicules et catégories
- Gérer utilisateurs et rôles
- Gérer les remboursements

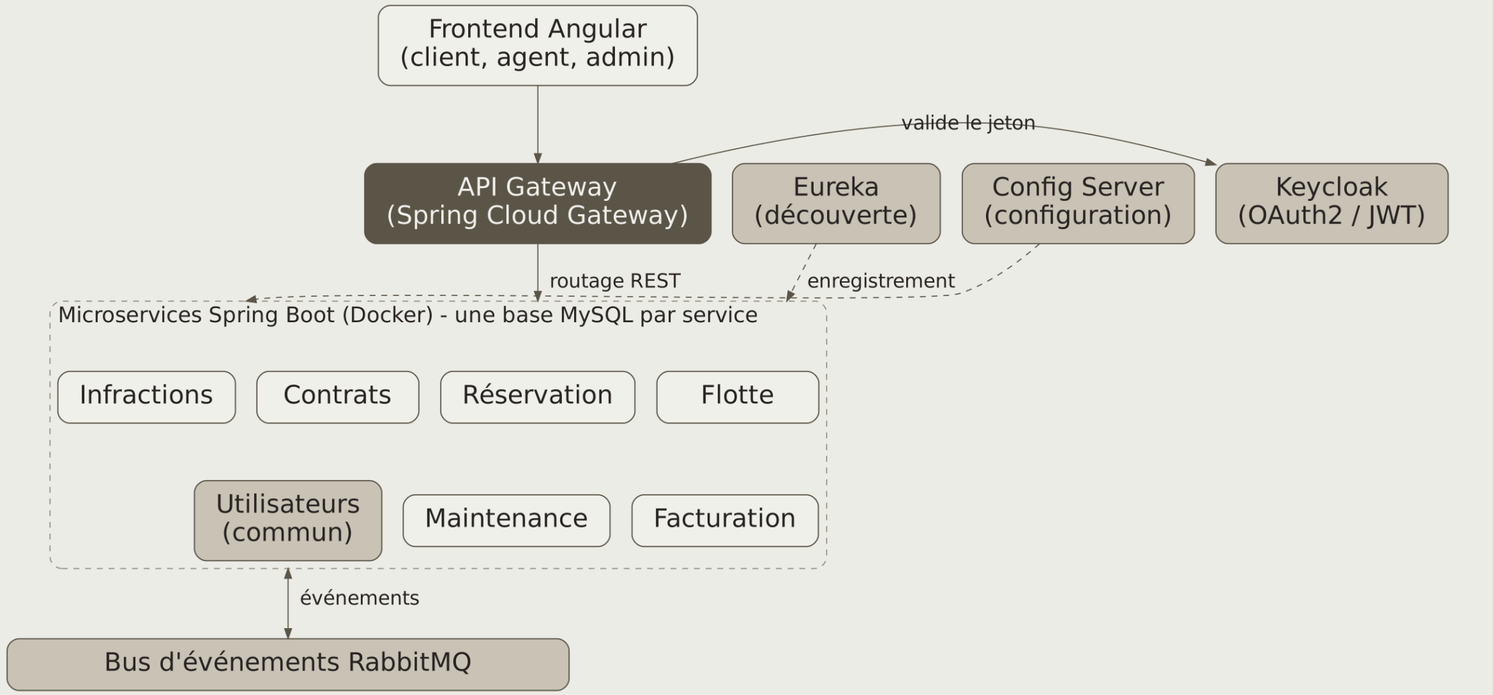
Fonctionnalité clé : déclarer une infraction et identifier automatiquement le conducteur grâce au contrat actif.

RENTIFY - VALIDATION 1 AWD

ARCHITECTURE GLOBALE

SERVICES, GATEWAY, SÉCURITÉ, DONNÉES





Accès : le frontend appelle uniquement l'API Gateway.

Sécurité : Keycloak (OAuth2, JWT), rôles CLIENT, AGENT, ADMIN.

Découverte et configuration : Eureka et Config Server.

Données : une base MySQL par service.

Communication : REST (OpenFeign) et événements RabbitMQ.

Déploiement : Docker Compose.

RENTIFY - VALIDATION 1 AWD

TECHNOLOGIES

STACK CHOISIE



MICROSERVICES

SPRING BOOT

API REST, OpenFeign, Resilience4j

POINT D'ENTRÉE

API GATEWAY

Spring Cloud Gateway

INFRASTRUCTURE

EUREKA

Découverte et Config Server

SÉCURITÉ

KEYCLOAK

OAuth2, JWT, rôles

DONNÉES

MYSQL

Une base par service

ÉVÉNEMENTS

RABBITMQ

Communication asynchrone

FRONTEND

ANGULAR

Applications client et agent

DÉPLOIEMENT

DOCKER

Docker Compose, un conteneur par service

RENTIFY - VALIDATION 1 AWD

MICROSERVICES

UN CONTEXTE MÉTIER = UN MICROSERVICE



Microservice	Responsabilité	Base de données	Responsable
Flotte	Véhicules, catégories, disponibilité	MySQL	Membre 1
Réservation	Réserver, modifier, annuler	MySQL	Membre 2
Contrats	Contrats, conducteurs, états des lieux	MySQL	Membre 3
Infractions	Déclarer, identifier le conducteur	MySQL	Membre 4
Facturation	Factures, paiements, pénalités	MySQL	Membre 5
Maintenance	Entretien et réparations	MySQL	Membre 6
Utilisateurs (commun)	Profils, permis, rôles (avec Keycloak)	MySQL	Toute l'équipe

Choix de MySQL : données relationnelles et transactions ACID pour les réservations et les paiements. Une base distincte par service, aucune base partagée.

RENTIFY - VALIDATION 1 AWD

COMMUNICATION

ENTRE LES MICROSERVICES



SYNCHRONE : REST AVEC OPENFEIGN

LECTURE IMMÉDIATE

- Réservation vers Flotte : disponibilité
- Contrats vers Réservation et Utilisateurs
- Infractions vers Flotte et Contrats
- Facturation et Maintenance vers Flotte

ASYNCHRONE : ÉVÉNEMENTS RABBITMQ

CHANGEMENT D'ÉTAT

- ReservationConfirmee, ReservationAnnulee
- ContratOuvert, ContratCloture
- ConducteurIdentifie
- InterventionPlanifiee, InterventionTerminee
- UtilisateurDesactive

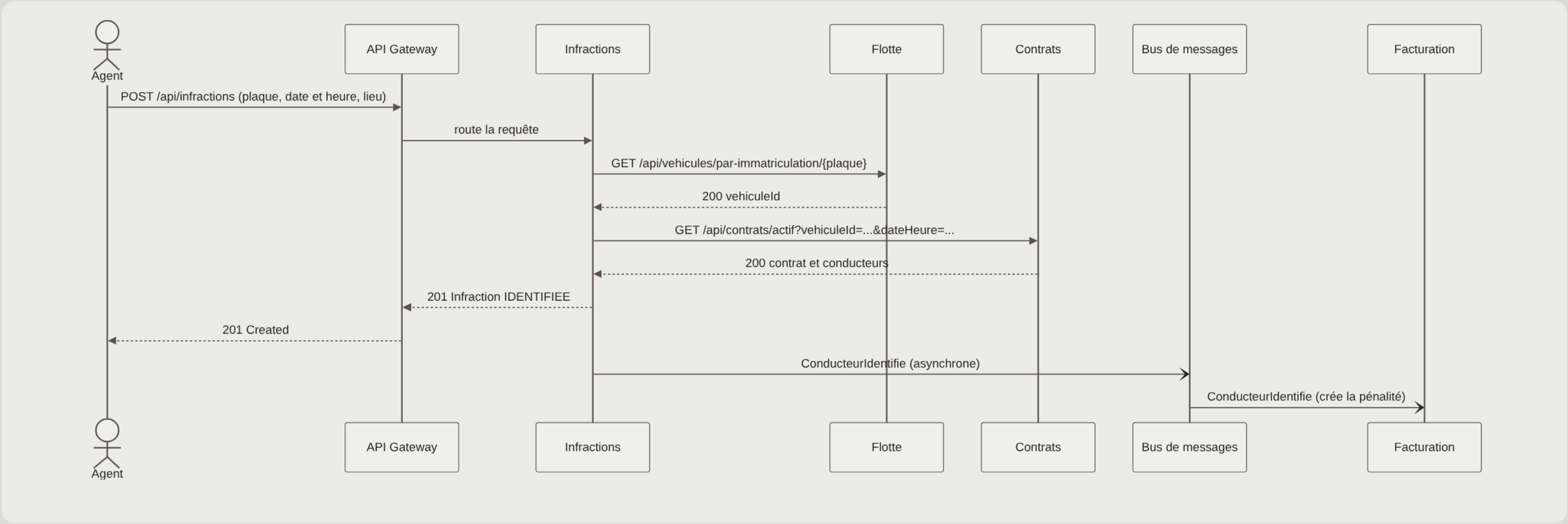
Service indisponible : timeout, retry et circuit breaker (Resilience4j) ; les événements sont idempotents.

RENTIFY - VALIDATION 1 AWD

SCÉNARIO : INFRACTION

IDENTIFIER LE CONDUCTEUR





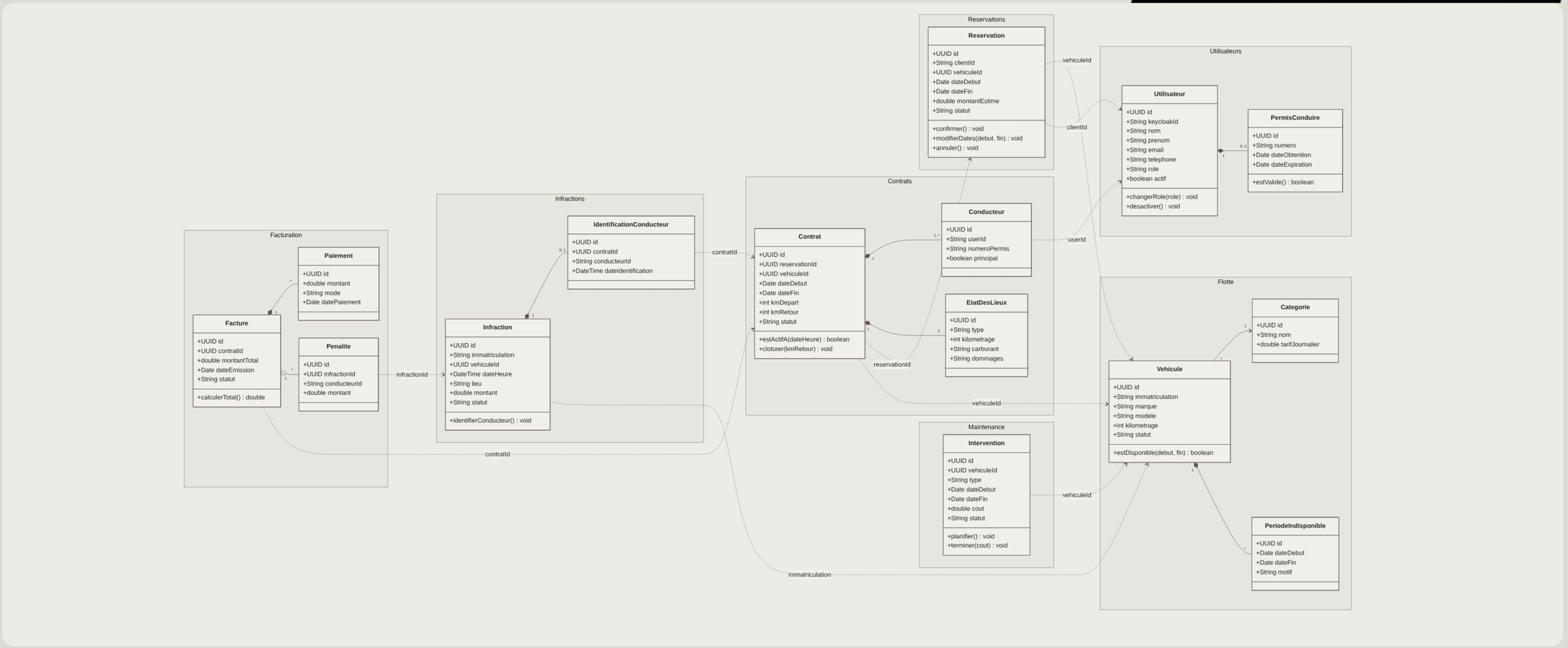
Synchrone pour répondre à l'agent tout de suite, **asynchrone** pour créer la pénalité sans bloquer.

RENTIFY - VALIDATION 1 AWD

DIAGRAMME DE CLASSES

GLOBAL : 7 MICROSERVICES, LIENS PAR IDENTIFIANT



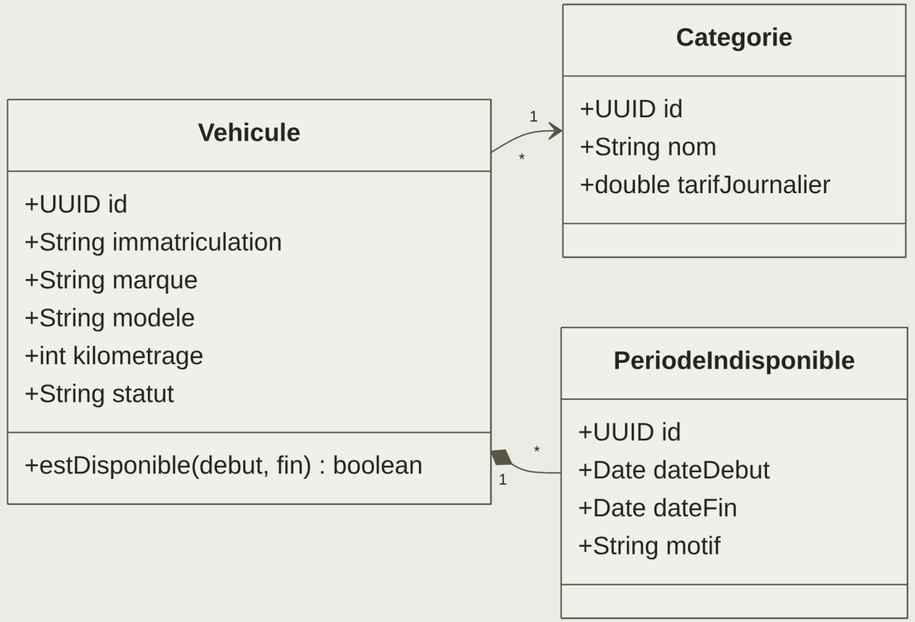


RENTIFY - VALIDATION 1 AWD

FLOTTE

MEMBRE 1 - RESPONSABLE DU MICROSERVICE





Méthode	Endpoint	Codes
GET	/api/vehicules/disponibles?debut=&fin=	200, 400
GET	/api/vehicules/{id}/disponibilite	200, 404
GET	/api/vehicules/par-immatriculation/{plaque}	200, 404
POST	/api/vehicules	201, 400, 403
PATCH	/api/vehicules/{id}/statut	200, 404, 409

Événements consommés : ReservationConfirmee/Annulee, ContratOuvert/Cloture, InterventionPlanifiee/Terminee. **Publiés :** aucun.

Appelé par : Réservation, Infractions, Facturation, Maintenance.

RENTIFY - VALIDATION 1 AWD

RÉSERVATION

MEMBRE 2 - RESPONSABLE DU MICROSERVICE



Reservation
+UUID id +String clientId +UUID vehiculeId +Date dateDebut +Date dateFin +double montantEstime +String statut
+confirmer() : void +modifierDates(debut, fin) : void +annuler() : void

Méthode	Endpoint	Codes
POST	/api/reservations	201, 400, 409
GET	/api/reservations/{id}	200, 404
PUT	/api/reservations/{id}	200, 400, 409
DELETE	/api/reservations/{id}	204, 404, 409

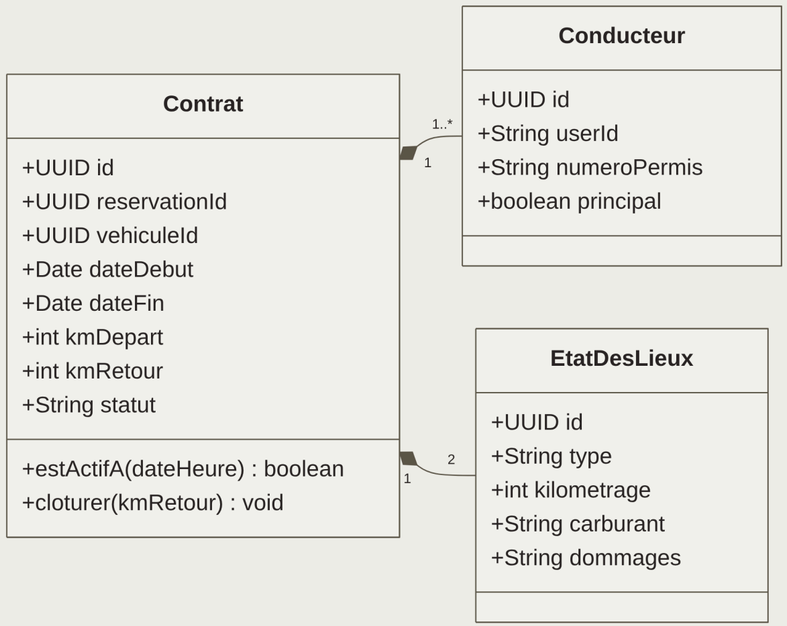
Événements publiés : ReservationConfirmee, ReservationAnnulee. **Consommé :** UtilisateurDesactive.
Appel synchrone : Flotte (disponibilité du véhicule). **Appelé par :** Contrats.

RENTIFY - VALIDATION 1 AWD

CONTRATS

MEMBRE 3 - RESPONSABLE DU MICROSERVICE





Méthode	Endpoint	Codes
POST	/api/contrats	201, 400, 404, 409
GET	/api/contrats/actif?vehiculeId=&dateHeure=	200, 404
POST	/api/contrats/{id}/conducteurs	201, 400, 404
POST	/api/contrats/{id}/retour	200, 404, 409

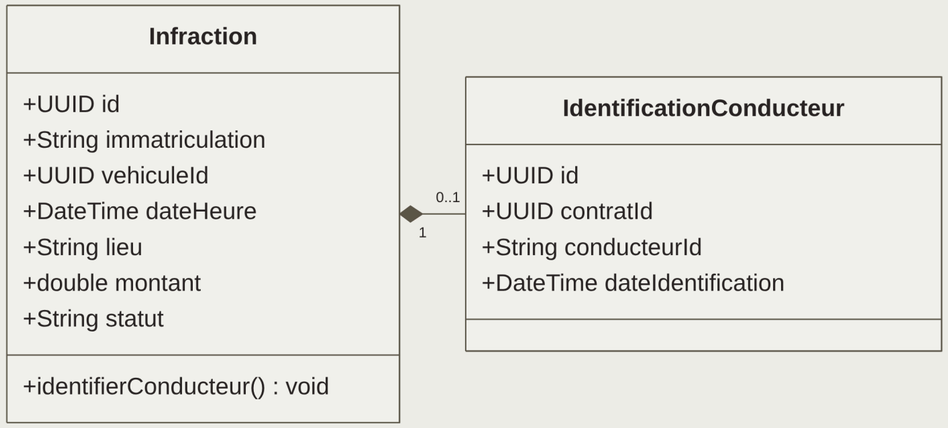
Événements publiés : ContratOuvert, ContratCloture.
Appels synchrones : Réservation, Utilisateurs (validité du permis). **Appelé par :** Infractions.

RENTIFY - VALIDATION 1 AWD

INFRACTIONS

MEMBRE 4 - RESPONSABLE DU MICROSERVICE





Méthode	Endpoint	Codes
POST	/api/infractions	201, 400, 404
GET	/api/infractions/{id}/conducteur	200, 404
POST	/api/infractions/{id}/identification	200, 404, 409
PATCH	/api/infractions/{id}/conducteur	200, 400, 404

Événement publié : ConducteurIdentifie.

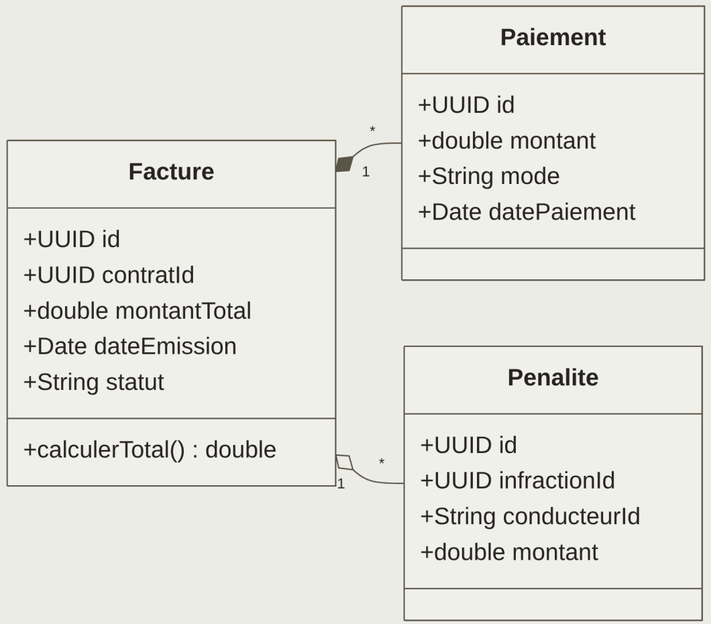
Appels synchrones : Flotte (plaque vers véhicule), Contrats (contrat actif à la date).

RENTIFY - VALIDATION 1 AWD

FACTURATION

MEMBRE 5 - RESPONSABLE DU MICROSERVICE





Méthode	Endpoint	Codes
GET	/api/factures/{id}	200, 404
GET	/api/factures/par-contrat/{contratId}	200, 404
POST	/api/factures/{id}/paiements	201, 400, 404, 409
POST	/api/factures/{id}/remboursement	200, 404, 409

Événements consommés : ContratCloture (crée la facture), ConducteurIdentifie (crée la pénalité). **Publiés :** aucun.
Appel synchrone : Flotte (tarif).

RENTIFY - VALIDATION 1 AWD

MAINTENANCE

MEMBRE 6 - RESPONSABLE DU MICROSERVICE



Intervention
+UUID id +UUID vehiculeId +String type +Date dateDebut +Date dateFin +double cout +String statut
+planifier() : void +terminer(cout) : void

Méthode	Endpoint	Codes
POST	/api/interventions	201, 400, 404, 409
GET	/api/interventions?vehiculeId=	200
PATCH	/api/interventions/{id}/terminer	200, 404, 409

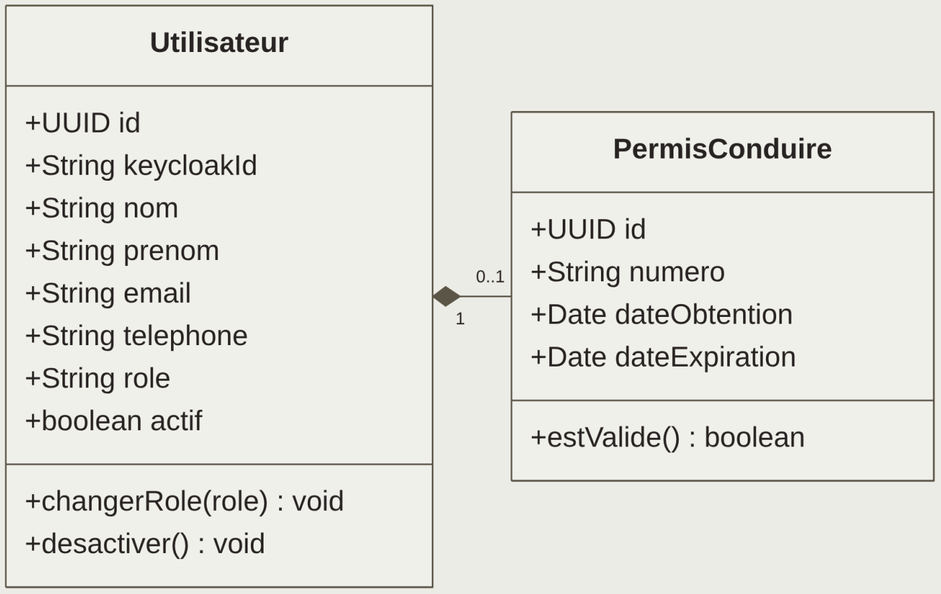
Événements publiés : InterventionPlanifiee, InterventionTerminee. **Consommé :** ContratCloture.
Appel synchrone : Flotte (existence du véhicule).

RENTIFY - VALIDATION 1 AWD

UTILISATEURS

MICROSERVICE COMMUN - PRÉSENTÉ PAR LE MEMBRE 6





Méthode	Endpoint	Codes
POST	/api/utilisateurs	201, 400, 409
GET	/api/utilisateurs/me	200, 401
GET	/api/utilisateurs/{id}	200, 403, 404
PUT	/api/utilisateurs/{id}	200, 400, 404
PATCH	/api/utilisateurs/{id}/role	200, 403, 404
PATCH	/api/utilisateurs/{id}/statut	200, 404

Événement publié : UtilisateurDesactive. **Appel synchrone :** API d'administration de Keycloak.
Appelé par : Contrats (validité du permis). Les autres services ne stockent que l'identifiant de l'utilisateur.

RENTIFY - VALIDATION 1 AWD

ORGANISATION

DÉPÔT GIT ET RÉPARTITION DES TÂCHES



DÉPÔT GIT PUBLIC

```
Rentify-4ssa9-g3/
├─ Backend/ (services + common-lib)
├─ Frontend/
├─ Documentation/
│  ├─ Presentation/
│  └─ Diagrams/
└─ README.md
```

Branches : main, develop, feature/<service>-<sujet>, avec pull requests relues. **Suivi :** GitHub Projects (Kanban).

Membre	Microservice	Autres tâches
Membre 1	Flotte	API Gateway
Membre 2	Réservation	Frontend réservation
Membre 3	Contrats	Frontend agent
Membre 4	Infractions	Configuration Keycloak
Membre 5	Facturation	Docker Compose
Membre 6	Maintenance	README, documentation
Équipe	Utilisateurs	Développement collectif, revue croisée

PLANNING ET RISQUES

PRÉVISIONNEL



Période	Livrables
Semaine 1	Conception, dépôt Git, présentation et document
Semaine 2	Infrastructure (Docker Compose, Keycloak, Gateway, Eureka, Config Server, RabbitMQ) et squelettes des services
Semaines 3 et 4	Fonctionnalités principales et frontend
Semaines 5 et 6	Événements, tests, sécurité, démonstration

Risque	Mesure
Données incohérentes entre services	Événements idempotents, statuts explicites
Service indisponible en appel synchrone	Timeouts, retry, circuit breaker
Intégration tardive	Contrats d'API figés dès la semaine 1, intégration continue
Charge inégale ou absence	Revue croisée, Kanban, tous maîtrisent l'ensemble

DES QUESTIONS ?

Merci



GITHUB.COM/SARA-B-GH/RENTIFY-4SSA9-G3

ÉQUIPE 4SSA9 - G3